COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 15, 2026
NAME:	Payas, John Paul S. Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)		
Strategy	**Algorithm**	**Analysis**
Recursion	Factorial	The function calls itself with a smaller input until it reaches the base case.
Brute Force	Linear Search	The algorithm checks each element one by one until the target value is found.
Backtracking	N-Queens	The algorithm tries possible queen positions and returns to an earlier position when a placement is invalid.
Greedy	Coin Change	The algorithm chooses the largest possible coin at each step. Then, it repeats the process as long as the coin value does not
Divide-and-Conquer	Binary Search	The algorithm repeatedly divides the search interval into smaller parts.

Table 12-1. Algorithmic Strategies and Examples

Screenshot

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int getMinSteps(int n, vector<int>& memo) {
    if (n == 1)
        return 0;

    if (memo[n] != -1)
        return memo[n];
    int result = 1 + getMinSteps(n - 1, memo);
    if (n % 2 == 0)
        result = min(result, 1 + getMinSteps(n / 2, memo));
    if (n % 3 == 0)
        result = min(result, 1 + getMinSteps(n / 3, memo));
    memo[n] = result;
    return result;
}
```

```cpp
int main() {
    int n;

    cout << "Enter a positive integer: ";
    cin >> n;
    if (n < 1) {
        cout << "Invalid input." << endl;
        return 0;
    }
    vector<int> memo(n + 1, -1);
    cout << "Minimum steps to reach 1: "
         << getMinSteps(n, memo) << endl;

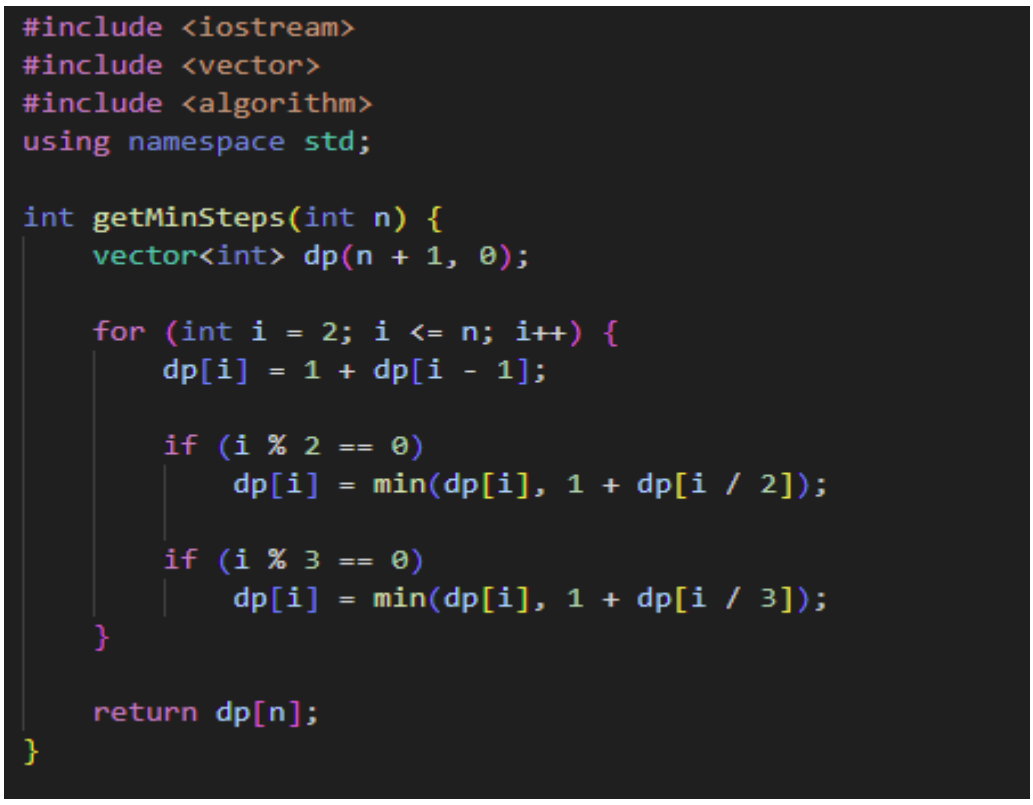
    return 0;
}
```

```
er.exe' '--stdin=Microsoft-MIEngine-In-tzvxrefr.zyn'
lhivukc.qrq' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe'
Enter a positive integer: 12
Minimum steps to reach 1: 3
PS C:\Users\TIPQC\Documents\HOA12p1_REYES>
```

```
t0tijdg.lpm' '--dbgExe=C:\msys64\ucrt64\b
Enter a positive integer: 500
Minimum steps to reach 1: 9
```

Analysis	**Theoretical Analysis:** The getMinSteps() function uses recursion with memoization in order to find the minimum number of operations needed to reduce a positive integer n to 1. For each value, it considers subtracting 1 and, when it is divisible, the algorithm then divides it by 2 or 3. Furthermore, the min() function selects the option requiring fewest steps, while the memo vector stores previously calculated results to avoid repeated computations. If we further analyze this, we can see that the time complexity is O(n) because each value from 2 to n is calculated at most once, with a constant number of operations per value. The space complexity is O(n) because the memoization vector stores up to n+1 values, and the recursive calls may also use up to O(n) stack space. **Experimental Analysis:** The program was tested using different inputs. For n = 10, it returned 3 steps, while n = 4 returned 2 steps and n = 1 returned 0 steps. These results show that the program correctly finds the minimum number of steps to reach 1.

Table 12-2. Memoization Implementation

Screenshot	

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int getMinSteps(int n) {
    vector<int> dp(n + 1, 0);

    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];

        if (i % 2 == 0)
            dp[i] = min(dp[i], 1 + dp[i / 2]);

        if (i % 3 == 0)
            dp[i] = min(dp[i], 1 + dp[i / 3]);
    }

    return dp[n];
}
```

```cpp
int main() {
    int n;

    cout << "Enter a positive integer: ";
    cin >> n;

    if (n < 1) {
        cout << "Invalid input." << endl;
        return 0;
    }

    cout << "Minimum steps to reach 1: "
         << getMinSteps(n) << endl;

    return 0;
}
```

```
Enter a positive integer: 5
Minimum steps to reach 1: 3
```

Analysis

Theoretical Analysis:

The program uses the bottom-up dynamic programming approach to find the minimum number of steps required to reduce a positive integer n to 1. It starts with the base case dp[1] = 0 and calculates the minimum steps for each number from 2 to n. For every number, it considers subtracting 1 and, when divisible, dividing by 2 or 3. The min() function selects the operation that results in the fewest steps. The theoretical time complexity is O(n) because each number is processed once, while the space complexity is O(n) because the DP array stores the results for all numbers from 1 to n.

Experimental Analysis:

When the program is executed with an input of 10, it displays a minimum of 3 steps to reach 1. The optimal sequence is $10 \rightarrow 9 \rightarrow 3 \rightarrow 1$, using subtraction by 1 followed by division by 3 twice. This result demonstrates that the program correctly finds the minimum number of steps instead of simply choosing the operation that reduces the number most immediately. The program can also handle other positive integer inputs, and its execution time and memory usage generally increase as the input value becomes larger.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

PSEUDOCODE (+ SOLVING PROBLEM BY HAND)

```
FUNCTION CountPaths(matrix, row, col, remainingCost)

   IF row or col is outside the matrix
      RETURN 0

   remainingCost = remainingCost - matrix[row][col]

   IF remainingCost < 0
      RETURN 0

   IF current cell is the destination
      IF remainingCost == 0
         RETURN 1
      ELSE
         RETURN 0

   rightPaths = CountPaths(matrix, row, col + 1, remainingCost)
   downPaths = CountPaths(matrix, row + 1, col, remainingCost)

   RETURN rightPaths + downPaths
END FUNCTION
```

Solution by Hand:

Given:

$$\begin{bmatrix} 4 & 7 & 1 & 6 \\ 6 & 7 & 3 & 9 \\ 3 & 8 & 1 & 2 \\ 7 & 1 & 7 & 3 \end{bmatrix}$$

- Start: (0,0)
- End: (3,3)
- Target cost: 25
- Moves: right or down
- Cost = Sum of all visited cells (including start and end)

Example Path:

Right, Right, Down, Down, Down, Right

4 →	7 →	1	→6
↓			
6	7	3	9
↓			
3	8	1	2
↓			
7	1	7	3

Path cost:

$4 + 7 + 1 + 3 + 1 + 7 + 3$

$= 26$

This path does not satisfy the target cost of 25.

Checking all possible paths:

- Each path has exactly 3 right moves and 3 down moves.
- After checking all valid paths, there are no paths with a total cost of 25.
- There are 2 paths with a total cost of 26.

Conclusion:

Number of paths with cost 25 = 0 (for given matrix)

Number of paths with cost 26 = 2

CODE

```cpp
#include <iostream>
#include <vector>
using namespace std;

int countPaths(const vector<vector<int>>& matrix,
               int row, int col, int remainingCost,
               vector<vector<vector<int>>>& memo) {
    int m = matrix.size();
    int n = matrix[0].size();

    if (row >= m || col >= n)
        return 0;

    if (matrix[row][col] > remainingCost)
        return 0;

    if (memo[row][col][remainingCost] != -1)
        return memo[row][col][remainingCost];

    remainingCost -= matrix[row][col];

    if (row == m - 1 && col == n - 1)
        return memo[row][col][remainingCost + matrix[row][col]]
            = (remainingCost == 0 ? 1 : 0);

    int right = countPaths(matrix, row, col + 1, remainingCost, memo);

    int down = countPaths(matrix, row + 1, col, remainingCost, memo);

    return memo[row][col][remainingCost + matrix[row][col]]
        = right + down;
}
```

```cpp
int main() {
    vector<vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost = 26;
    int m = matrix.size();
    int n = matrix[0].size();

    vector<vector<vector<int>>> memo(
        m, vector<vector<int>>(
            n, vector<int>(targetCost + 1, -1)
        )
    );

    cout << "Target cost: " << targetCost << endl;
    cout << "Number of valid paths: "
         << countPaths(matrix, 0, 0, targetCost, memo)
         << endl;

    return 0;
}
```

ANALYSIS OF WORKING CODE

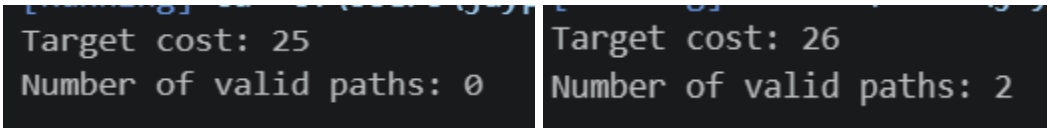
Theoretical Analysis:

The program uses recursion and dynamic programming with memoization to count the number of paths from the first cell to the destination. It explores two possible movements, right and down, and subtracts the value of each visited cell from the remaining cost. If a path exceeds the target cost or moves outside the matrix, it returns zero. The memoization table stores previously computed results to prevent repeated calculations. The theoretical time and space complexities are both O(MNK), where M is the number of rows, N is the number of columns, and K is the target cost

Experimental Analysis:

When the program runs with the given matrix and a target cost of 25, it returns zero valid paths. When the target cost is changed to 26, it returns two valid paths. These results demonstrate that the program checks the total cost of each possible path and counts only those that match the target. The experiment also demonstrates how memoization reduces repeated calculations and improves efficiency.

SCREENSHOTS OF DEMONSTRATION



CONCLUSION – PAYAS

In this activity, I learned about different algorithmic strategies, including recursion, brute force, backtracking, greedy algorithms, divide and conquer, and dynamic programming. I learned how to implement memoization and bottom-up to find the minimum number of steps needed to reach 1. I also applied recursion and memoization to count paths in a matrix with a given cost. Through the procedures, I understood the importance of selecting an appropriate algorithm, analyzing its time and space complexity, and testing the program to verify its results. The supplementary activity also teaches me to check the given data carefully and compare the expected output with the actual output.

CONCLUSION – REYES

In this activity, I was able to learn about various techniques that can be used in solving algorithmic problems using recursion, memoization, and dynamic programming bottom up approaches. Using the minimum steps problem, I learned the significance of keeping track of previous computations to avoid repetitive computation and come up with the most efficient solution. Moreover, using the additional activity, I learned how recursion and memoization can be used to count the number of valid paths in a matrix depending on the cost. I was able to learn about the importance of verifying program outputs as well as analyzing the time and space complexities of an algorithm. Overall, this activity increased my understanding of how to design algorithms and solve problems in C++. However, there is still much that I need to learn especially when it comes to coming up with algorithms independently and analyzing their efficiency.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of Concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only paved the way to explain concepts and helped us analyze concepts in the HOA ILOs guidelines in order for us to meet the required output. Other than that, it did not replace our original idea and how we want our output to come up.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.